

优极限

“极限教育，挑战极限”

www.yjxxt.com

极限教育，挑战极限。优极限是一个让 95% 的学生年薪过 18 万的岗前培训公司，让我们的学员具备优秀的互联网技术和职业素养，勇攀高薪，挑战极限。公司位于上海浦东，拥有两大校区，共万余平。累计培训学员超 3 万名。我们的训练营就业平均月薪 19000，最高年薪 50 万。

核心理念：让学员学会学习，拥有解决问题的能力，拿到高薪职场的钥匙。

项目驱动式团队协作、一对一服务、前瞻性思维、教练式培养模型-培养你成为就业明星。首创的老学员项目联盟给学员充分的项目、技术支撑，利用优极限平台这根杠杆，不断挑战极限，勇攀高薪，开挂人生。

扫码关注优极限微信公众号：

（获取最新技术相关资讯及更多源码笔记）



SpringBoot - 第二天

1. 学习目标



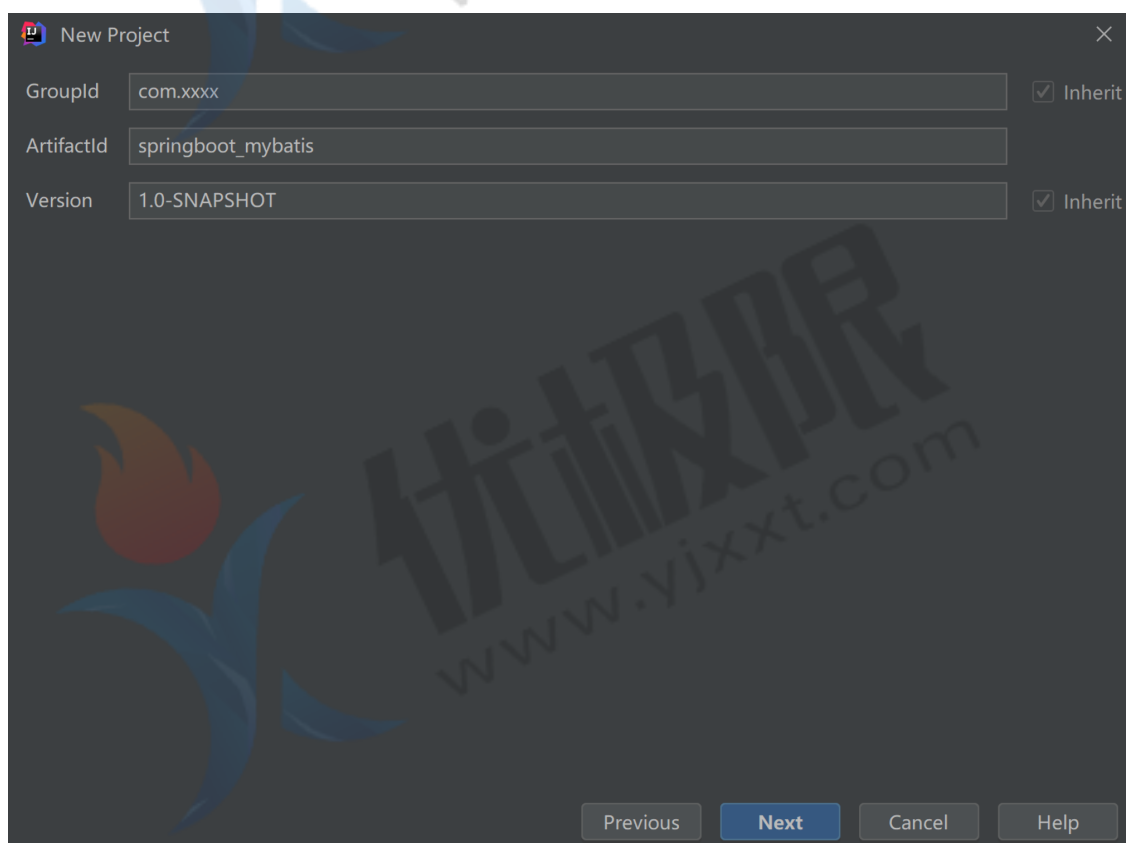
2. Mybatis 整合 & 数据访问

使用 SpringBoot 开发企业项目时，持久层数据访问是前端页面数据展示的基础，SpringBoot 支持市面上常见的关系库产品 (Oracle、Mysql、SqlServer、DB2等) 对应的相关持久层框架，当然除了对于关系库访问的支持，也支持当下众多的非关系库 (Redis、Solr、MongoDB等) 数据访问操作，这里主要介绍 SpringBoot 集成 Mybatis 并实现持久层数据基本增删改查操作。

2.1. SpringBoot 整合 Mybatis

2.1.1. 环境整合配置

- Idea 下创建 Maven 普通工程 springboot_mybatis



- pom.xml 添加核心依赖

```
<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <maven.compiler.source>11</maven.compiler.source>
  <maven.compiler.target>11</maven.compiler.target>
</properties>

<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>2.2.2.RELEASE</version>
</parent>

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <!-- mybatis 集成 -->
  <dependency>
    <groupId>org.mybatis.spring.boot</groupId>
    <artifactId>mybatis-spring-boot-starter</artifactId>
    <version>2.1.1</version>
  </dependency>
  <!-- springboot 分页插件 -->
  <dependency>
    <groupId>com.github.pagehelper</groupId>
    <artifactId>pagehelper-spring-boot-starter</artifactId>
    <version>1.2.13</version>
  </dependency>
  <!-- mysql 驱动 -->
  <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
  </dependency>
  <!-- c3p0 数据源 -->
  <dependency>
    <groupId>com.mchange</groupId>
    <artifactId>c3p0</artifactId>
    <version>0.9.5.5</version>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

- application.yml 整合配置

```
server:
  # 设置项目启动的端口号
```

```

port: 8080
# 设置项目的访问路径（上下文路径）
servlet:
    context-path: /springboot_mybatis

## 数据源配置
spring:
    datasource:
        type: com.mchange.v2.c3p0.ComboPooledDataSource
        driver-class-name: com.mysql.cj.jdbc.Driver
        url: jdbc:mysql://127.0.0.1:3306/springboot_mybatis?
        useUnicode=true&characterEncoding=utf8&serverTimezone=GMT%2B8
        username: root
        password: root

## mybatis 配置
mybatis:
    #映射文件的存放路径
    mapper-locations: classpath:/mappers/*.xml
    type-aliases-package: com.xxxx.springboot.po
    configuration:
        ## 下划线转驼峰配置
        map-underscore-to-camel-case: true

## pageHelper
pagehelper:
    helper-dialect: mysql

## 显示dao 执行sql语句
logging:
    level:
        com:
            xxx:
                springboot:
                    dao: debug

```

2.1.2. 源代码添加

- JavaBean 对象定义

```

package com.xxxx.springboot.po;

public class User {

    private Integer id;
    private String userName;
    private String userPwd;

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getUserName() {

```

```

        return userName;
    }

    public void setUserName(String userName) {
        this.userName = userName;
    }

    public String getUserPwd() {
        return userPwd;
    }

    public void setUserPwd(String userPwd) {
        this.userPwd = userPwd;
    }
}

```

- Dao层接口方法定义

com.xxxx.springboot.dao 包下创建 UserMapper.java 接口声明查询方法

```

package com.xxxx.springboot.dao;

import com.xxxx.springboot.po.User;

public interface UserMapper {
    // 根据用户名查询用户记录
    User queryUserByUserName(String userName);
}

```

- SQL映射文件添加

resources/mappers 目录下添加 UserMapper.xml 配置，查询 statement

```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd" >
<mapper namespace="com.xxxx.springboot.dao.UserMapper">
    <select id="queryUserByUserName" parameterType="string"
resultType="com.xxxx.springboot.po.User">
        select
            id,user_name,user_pwd
        from tb_user
        where user_name=#{userName}
    </select>
</mapper>

```

- 添加 service、controller 对应代码

UserService.java

```

@Service
public class UserService {

    @Autowired
    private UserMapper userMapper;

    public User queryUserByUserName(String userName){
        return userMapper.queryUserByUserName(userName);
    }
}

```

UserController.java

```

@RestController
public class UserController {

    @Resource
    private UserService userService;

    @GetMapping("user/{userName}")
    public User queryUserByUserName(@PathVariable String userName){
        return userService.queryUserByUserName(userName);
    }
}

```

- 添加应用启动入口

```

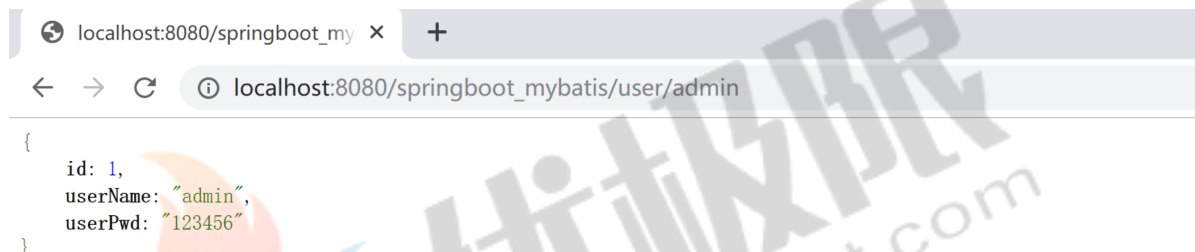
@SpringBootApplication
@MapperScan("com.xxxx.springboot.dao")
public class Starter {

    public static void main(String[] args) {
        SpringApplication.run(Starter.class);
    }
}

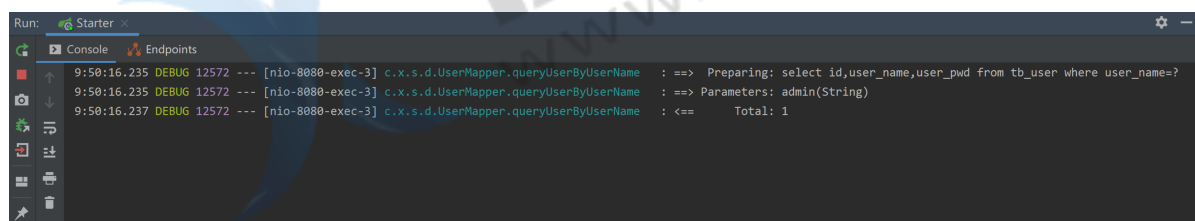
```

2.1.3. 启动测试

运行 Starter main方法，启动应用浏览器测试查询



后端日志打印效果：



2.2. SpringBoot 数据访问操作

完成 SpringBoot 与 Mybatis 集成后，接下来以用户表为例实现一套用户模块基本数据维护。

2.2.1. 查询操作

2.2.1.1. 接口方法定义

UserMapper 接口添加查询的方法

```
public interface UserMapper {  
  
    // 通过用户ID查询用户  
    public User queryById(Integer id);  
  
}
```

2.2.1.2. 映射文件配置

```
<?xml version="1.0" encoding="UTF-8" ?>  
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"  
"http://mybatis.org/dtd/mybatis-3-mapper.dtd" >  
<mapper namespace="com.xxxx.springboot.dao.UserMapper">  
    <select id="queryById" parameterType="int"  
        resultType="com.xxxx.springboot.po.User">  
        select  
            *  
        from  
            tb_user  
        where  
            id = #{id,jdbcType=INTEGER}  
    </select>  
  
</mapper>
```

2.2.1.3. UserService

```
@Service  
public class UserService {  
  
    @Resource  
    private UserMapper userMapper;  
  
    /**  
     * 通过用户ID查询用户  
     * @param id  
     * @return  
     */  
    public User queryById(Integer id){  
        return userMapper.queryById(id);  
    }  
  
}
```

2.2.1.4. UserController

```
@RestController
public class UserController {

    @Resource
    private UserService userService;

    /**
     * 根据用户ID查询用户对象
     * @param userId
     * @return
     */
    @GetMapping("user/{userId}")
    public User queryUserById(@PathVariable Integer userId){
        return userService.queryById(userId);
    }
}
```

2.2.2. 添加操作

2.2.2.1. 接口方法定义

```
public interface UserMapper {

    // 添加用户
    public int save(User user);

}
```

2.2.2.2. 映射文件配置

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd" >
<mapper namespace="com.xxxx.springboot.dao.UserMapper">

    <insert id="save" parameterType="com.xxxx.springboot.po.User">
        insert into
            tb_user
            (user_name,user_pwd)
        values
            (#{userName},#{userPwd})
    </insert>

</mapper>
```

2.2.2.3. 添加 commons-lang3 依赖

如果需要使用 StringUtils 工具类，需要引入 commons-lang3 依赖。

```
<dependency>
    <groupId>org.apache.commons</groupId>
    <artifactId>commons-lang3</artifactId>
</dependency>
```

2.2.2.4. AssertUtil 工具类

```
package com.xxxx.springboot.utils;

import com.xxxx.springboot.exceptions.ParamsException;

public class AssertUtil {

    /**
     * 判断结果是否为true
     * 如果结果为true, 抛出异常
     * @param userName
     * @return
     */
    public static void isTrue(Boolean flag, String msg){
        if(flag){
            throw new ParamsException(msg);
        }
    }

}
```

2.2.2.5. ParamsException 自定义异常

```
package com.xxxx.springboot.exceptions;

/**
 * 自定义参数异常
 */
public class ParamsException extends RuntimeException {
    private Integer code = 300;
    private String msg = "参数异常!";

    public ParamsException() {
        super("参数异常!");
    }

    public ParamsException(String msg) {
        super(msg);
        this.msg = msg;
    }

    public ParamsException(Integer code) {
        super("参数异常!");
        this.code = code;
    }

    public ParamsException(Integer code, String msg) {
        super(msg);
        this.code = code;
    }
}
```

```

        this.msg = msg;
    }

    public Integer getCode() {
        return code;
    }

    public void setCode(Integer code) {
        this.code = code;
    }

    public String getMsg() {
        return msg;
    }

    public void setMsg(String msg) {
        this.msg = msg;
    }
}

```

2.2.2.6. UserService

```

@Service
public class UserService {

    @Resource
    private UserMapper userMapper;

    /**
     * 添加用户
     * @param user
     */
    public void saveUser(User user) {
        AssertUtil.isTrue(StringUtils.isBlank(user.getUserName()), "用户名不能为空!");
        AssertUtil.isTrue(StringUtils.isBlank(user.getUserPwd()), "用户密码不能为空!");
        User temp = userMapper.queryUserByUserName(user.getUserName());
        AssertUtil.isTrue(null != temp, "该用户已存在!");
        AssertUtil.isTrue(userMapper.save(user) < 1, "用户记录添加失败!");
    }
}

```

2.2.2.7. ResultInfo

```

package com.xxxx.springboot.po.vo;

public class ResultInfo {

    private Integer code = 200;
    private String msg = "操作成功";
    private Object result;

    public Integer getCode() {
        return code;
    }
}

```

```

    }

    public void setCode(Integer code) {
        this.code = code;
    }

    public String getMsg() {
        return msg;
    }

    public void setMsg(String msg) {
        this.msg = msg;
    }

    public Object getResult() {
        return result;
    }

    public void setResult(Object result) {
        this.result = result;
    }
}

```

2.2.2.8. UserController

```

@RestController
public class UserController {

    @Resource
    private UserService userService;

    /**
     * 添加用户
     * @param user
     * @return
     */
    @PostMapping("user")
    public ResultInfo saveUser(@RequestBody User user){
        ResultInfo resultInfo = new ResultInfo();
        try {
            userService.saveUser(user);
        } catch (ParamsException e) {
            e.printStackTrace();
            resultInfo.setCode(e.getCode());
            resultInfo.setMsg(e.getMsg());
        } catch (Exception e) {
            e.printStackTrace();
            resultInfo.setCode(300);
            resultInfo.setMsg("记录添加失败!");
        }
        return resultInfo;
    }
}

```

2.2.3. 修改操作

2.2.3.1. 接口方法定义

```
public interface UserMapper {  
  
    // 修改用户  
    public int update(User user);  
  
}
```

2.2.3.2. 映射文件配置

```
<?xml version="1.0" encoding="UTF-8" ?>  
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"  
"http://mybatis.org/dtd/mybatis-3-mapper.dtd" >  
<mapper namespace="com.xxxx.springboot.dao.UserMapper">  
    <update id="update" parameterType="com.xxxx.springboot.po.User">  
        update  
            tb_user  
        set  
            user_name =#{userName},user_pwd=#{userPwd}  
        where  
            id = #{id}  
    </update>  
  
</mapper>
```

2.2.3.3. UserService

```
@Service  
public class UserService {  
  
    @Resource  
    private UserMapper userMapper;  
  
    /**  
     * 修改用户  
     * @param user  
     */  
    public void updateUser(User user) {  
        AssertUtil.isTrue(StringUtils.isBlank(user.getUserName()), "用户名不能为空!");  
        AssertUtil.isTrue(StringUtils.isBlank(user.getUserPwd()), "用户密码不能为空!");  
        User temp = userMapper.queryUserByUserName(user.getUserName());  
        AssertUtil.isTrue(null != temp && !(temp.getId().equals(user.getId())),  
            "该用户已存在!");  
        AssertUtil.isTrue(userMapper.update(user) < 1, "用户记录添加失败!");  
    }  
  
}
```

2.2.3.4. UserController

```
@RestController
public class UserController {

    @Resource
    private UserService userService;

    /**
     * 修改用户
     * @param user
     * @return
     */
    @PostMapping("user")
    public ResultInfo updateUser(@RequestBody User user){
        ResultInfo resultInfo = new ResultInfo();
        try {
            userService.updateUser(user);
        } catch (ParamsException e) {
            e.printStackTrace();
            resultInfo.setCode(e.getCode());
            resultInfo.setMsg(e.getMsg());
        } catch (Exception e) {
            e.printStackTrace();
            resultInfo.setCode(300);
            resultInfo.setMsg("记录更新失败!");
        }
        return resultInfo;
    }
}
```

2.2.4. 删除操作

2.2.4.1. 接口方法定义

```
public interface UserMapper {

    // 删除用户
    public int deleteUserById(Integer id);

}
```

2.2.4.2. 映射文件配置

```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd" >
<mapper namespace="com.xxxx.springboot.dao.UserMapper">
    <delete id="deleteUserById" parameterType="int">
        delete from
            tb_user
        where
            id=#{id}
    </delete>
</mapper>

```

2.2.4.3. UserService

```

@Service
public class UserService {

    @Resource
    private UserMapper userMapper;

    /**
     * 删除用户
     * @param id
     */
    public void deleteUser(Integer id){

        AssertUtil.isTrue(null == id || null == userMapper.queryById(id), "待删除记录不存在!");
        AssertUtil.isTrue(userMapper.deleteUserById(id) < 1, "用户删除失败!");
    }

}

```

2.2.4.4. UserController

```

@RestController
public class UserController {

    @Resource
    private UserService userService;

    /**
     * 删除用户
     * @param userId
     * @return
     */
    @DeleteMapping("user/{userId}")
    public ResultInfo deleteUser(@PathVariable Integer userId){
        ResultInfo resultInfo = new ResultInfo();
        try {
            userService.deleteUser(userId);
        } catch (ParamsException e) {
            e.printStackTrace();
            resultInfo.setCode(e.getCode());
            resultInfo.setMsg(e.getMsg());
        }
    }
}

```

```
    }catch (Exception e) {  
        e.printStackTrace();  
        resultInfo.setCode(300);  
        resultInfo.setMsg("记录删除失败!");  
    }  
    return resultInfo;  
}  
}
```

2.2.5. 分页条件查询操作

2.2.5.1. UserQuery

```
package com.xxxx.springboot.query;  
  
public class UserQuery {  
  
    private Integer pageNum = 1; // 当前页  
    private Integer pageSize = 10; // 每页显示的数量  
    private String userName; // 查询条件: 用户名  
  
    public Integer getPageNum() {  
        return pageNum;  
    }  
  
    public void setPageNum(Integer pageNum) {  
        this.pageNum = pageNum;  
    }  
  
    public Integer getPageSize() {  
        return pageSize;  
    }  
  
    public void setPageSize(Integer pageSize) {  
        this.pageSize = pageSize;  
    }  
  
    public String getUserName() {  
        return userName;  
    }  
  
    public void setUserName(String userName) {  
        this.userName = userName;  
    }  
}
```

2.2.5.2. 接口方法定义

```
public interface UserMapper {  
  
    // 通过条件，分页查询用户列表  
    public List<User> selectByParams(UserQuery userQuery);  
  
}
```

2.2.5.3. 映射文件配置

```
<?xml version="1.0" encoding="UTF-8" ?>  
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"  
"http://mybatis.org/dtd/mybatis-3-mapper.dtd" >  
<mapper namespace="com.xxxx.springboot.dao.UserMapper">  
    <select id="selectByParams"  
        parameterType="com.xxxx.springboot.query.UserQuery"  
        resultType="com.xxxx.springboot.po.User">  
        select  
            *  
        from  
            tb_user  
        <where>  
            <if test="null != userName and userName != ''">  
                and user_name like concat('%',#{userName},'%')  
            </if>  
        </where>  
    </select>  
  
</mapper>
```

2.2.5.4. UserService

```
@Service  
public class UserService {  
  
    @Resource  
    private UserMapper userMapper;  
  
    /**  
     * 通过指定参数，分页查询用户列表  
     * @param userQuery  
     * @return  
     */  
    public PageInfo<User> queryUserByParams(UserQuery userQuery) {  
        PageHelper.startPage(userQuery.getPageNum(), userQuery.getPageSize());  
        return new PageInfo<User>(userMapper.selectByParams(userQuery));  
    }  
  
}
```

2.2.5.5. UserController

```
@RestController
public class UserController {

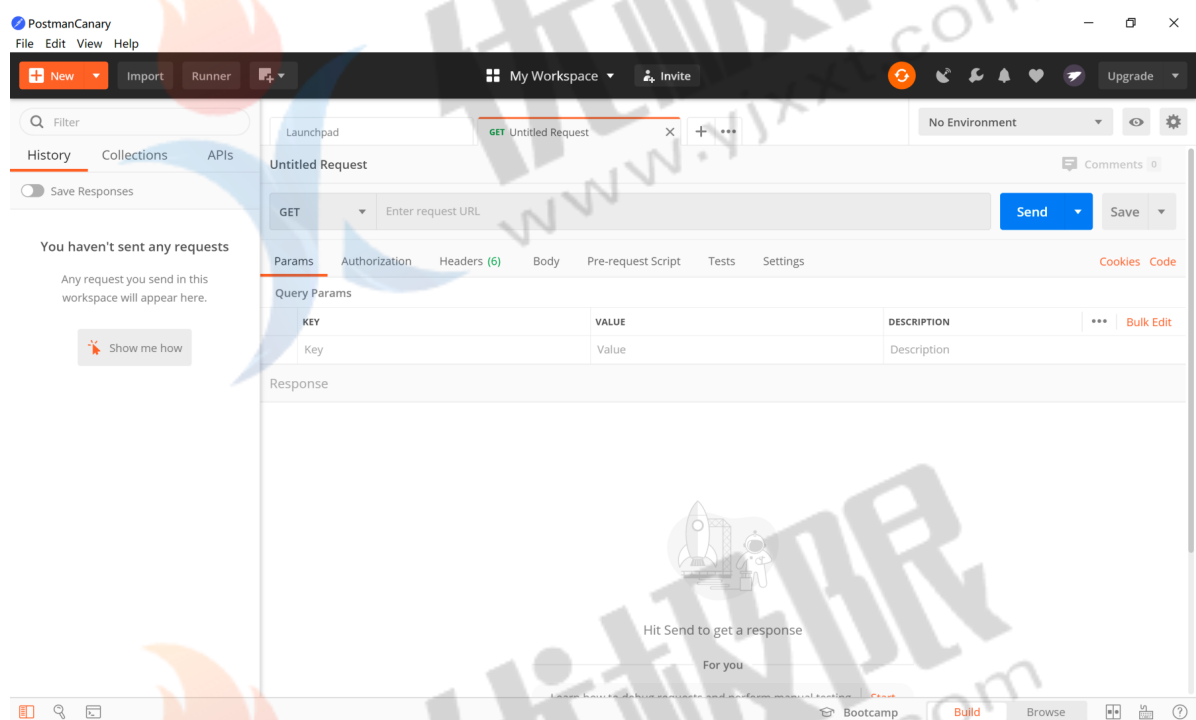
    @Resource
    private UserService userService;

    /**
     * 通过指定参数，分页查询用户列表
     * @param userQuery
     * @return
     */
    @GetMapping("user/list")
    public PageInfo<User> list(UserQuery userQuery){
        return userService.queryUserByParams(userQuery);
    }
}
```

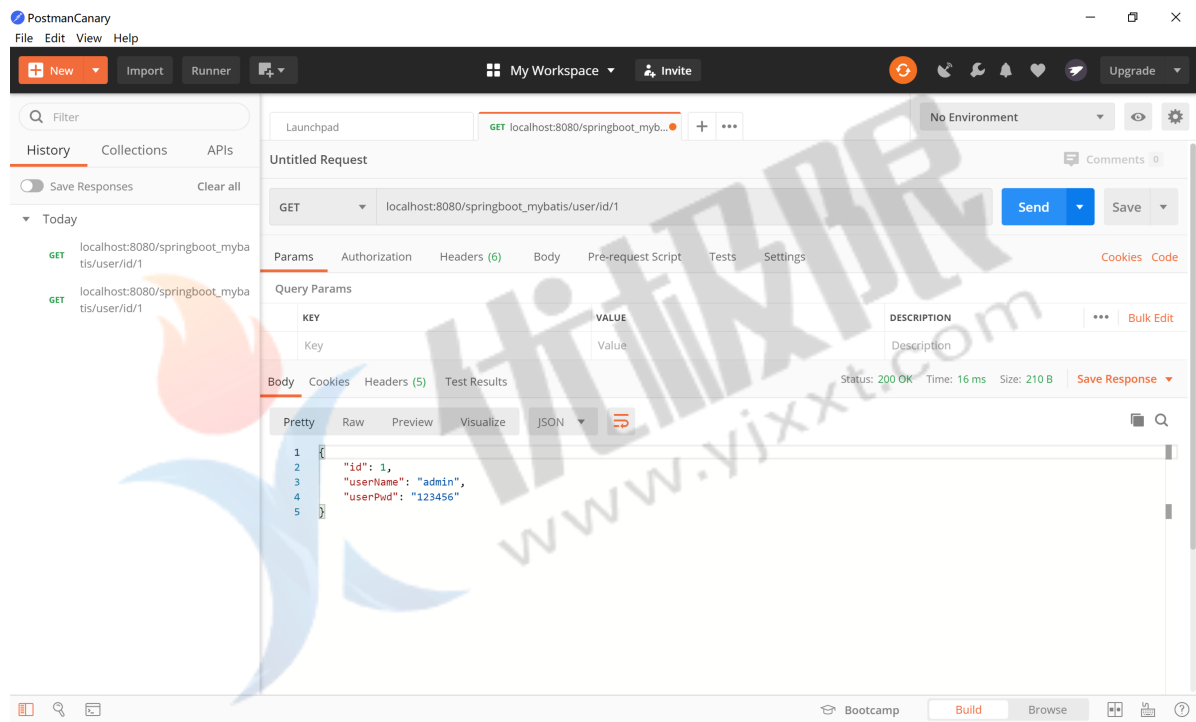
2.3. PostMan 工具下载与使用

在企业 web 应用开发中，对服务器端接口进行测试，通常借助接口测试工具，这里使用 Postman 接口测试工具来对后台 restful 接口进行测试。

Postman 工具下载地址：<https://www.getpostman.com/apps>，选中对应平台下载即可。



下载安装后，启动 Postman 根据后台接口地址发送响应请求即可对接口进行测试。



3. API 文档构建工具 - Swagger2

由于 Spring Boot 能够快速开发、便捷部署等特性，通常在使用 Spring Boot 构建 Restful 接口应用时考虑到多终端的原因，这些终端会共用很多底层业务逻辑，因此我们会抽象出这样一层来同时服务于多个移动端或者Web 前端。对于不同的终端公用一套接口 API 时，对于联调测试的时候就需要知道后端提供的接口 API列表文档，对于服务端开发人员来说就需要编写接口文档，描述接口的调用地址、参数结果等，这里借助第三方构建工具 Swagger2 来实现 API 文档生成功能。

3.1. 环境整合配置

- pom.xml 依赖添加

```
<dependency>
  <groupId>io.springfox</groupId>
  <artifactId>springfox-swagger2</artifactId>
  <version>2.9.2</version>
</dependency>
<dependency>
  <groupId>io.springfox</groupId>
  <artifactId>springfox-swagger-ui</artifactId>
  <version>2.9.2</version>
</dependency>
```

- 配置类添加

```
@Configuration
@EnableSwagger2
public class Swagger2 {
    @Bean
    public Docket createRestApi() {
        return new Docket(DocumentationType.SWAGGER_2)
            .apiInfo(apiInfo())
    }
}
```

```

        .select()

        .apis(RequestHandlerSelectors.basePackage("com.xxxx.springboot.controller"))
            .paths(PathSelectors.any())
            .build();
    }
    private ApiInfo apiInfo() {
        return new ApiInfoBuilder()
            .title("用户管理接口API文档")
            .version("1.0")
            .build();
    }
}

```

3.2. Swagger2 常用注解说明

3.2.1. @Api

@Api: 用在请求的类上，说明该类的作用
tags="说明该类的作用"

```
@Api(tags="APP用户注册Controller")
```

3.2.2. @ApiOperation

@ApiOperation: "用在请求的方法上，说明方法的作用"
value="说明方法的作用"
notes="方法的备注说明"

```
@ApiOperation(value="用户注册",notes="手机号、密码都是必填项，年龄是选填项，但必须是数字")
```

3.2.3. @ApiImplicitParams

@ApiImplicitParams: 用在请求的方法上，包含一组参数说明

@ApiImplicitParam: 用在 **@ApiImplicitParams** 注解中，指定一个请求参数的配置信息

name: 参数名

value: 参数的汉字说明、解释

required: 参数是否必须传

paramType: 参数放在哪个地方

- **header** --> 请求参数的获取: @RequestHeader
- **query** --> 请求参数的获取: @RequestParam
- **path** (用于restful接口) --> 请求参数的获取: @PathVariable
- **body** (不常用)
- **form** (不常用)

dataType: 参数类型，默认String，其它值**dataType**="Integer"

defaultValue: 参数的默认值

```

@ApiImplicitParams({
    @ApiImplicitParam(name="mobile",value="手机号",required=true,paramType="form"),
    @ApiImplicitParam(name="password",value="密码",required=true,paramType="form"),
    @ApiImplicitParam(name="age",value="年龄",required=true,paramType="form",dataType="Integer")
})

```

3.2.4. @ApiResponses

@ApiResponses: 用于请求的方法上，表示一组响应

@ApiResponse: 用在@ApiResponses中，一般用于表达一个错误的响应信息

code: 数字，例如400

message: 信息，例如"请求参数没填好"

response: 抛出异常的类

```

@ApiOperation(value = "select请求", notes = "多个参数，多种的查询参数类型")
@ApiResponses({
    @ApiResponse(code=400, message="请求参数没填好"),
    @ApiResponse(code=404, message="请求路径没有或页面跳转路径不对")
})

```

3.2.5. @ApiModel

@ApiModel: 用于响应类上，表示一个返回响应数据的信息
(这种一般用在post创建的时候，使用@RequestBody这样的场景，请求参数无法使用@ApiImplicitParam注解进行描述的时候)

@ApiModelProperty: 用在属性上，描述响应类的属性

```

@ApiModel(description= "返回响应数据")
public class RestMessage implements Serializable{
    @ApiModelProperty(value = "是否成功")
    private boolean success=true;
    @ApiModelProperty(value = "返回对象")
    private Object data;
    @ApiModelProperty(value = "错误编号")
    private Integer errCode;
    @ApiModelProperty(value = "错误信息")
    private String message;
    /* getter/setter */
}

```

3.3. 用户模块注解配置

3.3.1. Controller 使用注解

```

@GetMapping("user/{userName}")
@ApiOperation(value = "根据用户名查询用户记录")
@ApiImplicitParam(name = "userName",value = "查询参数",required = true,paramType = "path")
public User queryUserByUserName(@PathVariable String userName){
    return userService.queryUserByUserName(userName);
}

```

```
}

@ApiOperation(value = "根据用户id查询用户记录")
@ApiImplicitParam(name = "userId",value = "查询参数",required = true,paramType = "path")
@GetMapping("user/id/{userId}")
public User queryUserById(@PathVariable Integer userId, HttpServletRequest request){
    return userService.queryUserById(userId);
}

@GetMapping("user/list")
@ApiOperation(value = "多条件查询用户列表记录")
public PageInfo<User> list(UserQuery userQuery){
    return userService.queryUserByParams(userQuery);
}

@PutMapping("user")
@ApiOperation(value = "用户添加")
@ApiImplicitParam(name = "user",value = "用户实体类",dataType = "User")
public ResultInfo saveUser(@RequestBody User user){
    ResultInfo resultInfo=new ResultInfo();
    try {
        userService.saveUser(user);
    } catch (ParamsException e) {
        e.printStackTrace();
        resultInfo.setCode(e.getCode());
        resultInfo.setMsg(e.getMsg());
    }catch (Exception e) {
        e.printStackTrace();
        resultInfo.setCode(300);
        resultInfo.setMsg("记录添加失败!");
    }
    return resultInfo;
}

@PostMapping("user")
@ApiOperation(value = "用户更新")
@ApiImplicitParam(name = "user",value = "用户实体类",dataType = "User")
public ResultInfo updateUser(@RequestBody User user){
    ResultInfo resultInfo=new ResultInfo();
    try {
        userService.updateUser(user);
    } catch (ParamsException e) {
        e.printStackTrace();
        resultInfo.setCode(e.getCode());
        resultInfo.setMsg(e.getMsg());
    }catch (Exception e) {
        e.printStackTrace();
        resultInfo.setCode(300);
        resultInfo.setMsg("记录更新失败!");
    }
    return resultInfo;
}

@PutMapping("user/{userId}")
@ApiOperation(value = "根据用户id删除用户记录")
```

```

@ApiImplicitParam(name = "userId",value = "查询参数",required = true,paramType = "path")
public ResultInfo deleteUser(@PathVariable Integer userId){
    ResultInfo resultInfo=new ResultInfo();
    try {
        userService.deleteUser(userId);
    } catch (ParamsException e) {
        e.printStackTrace();
        resultInfo.setCode(e.getCode());
        resultInfo.setMsg(e.getMsg());
    }catch (Exception e) {
        e.printStackTrace();
        resultInfo.setCode(300);
        resultInfo.setMsg("记录删除失败!");
    }
    return resultInfo;
}

```

3.3.2. JavaBean 使用注解

- User.java

```

@ApiModel(description = "响应结果-用户信息")
public class User {
    @ApiModelProperty(value = "用户id",example = "0")
    private Integer id;
    @ApiModelProperty(value = "用户名")
    private String userName;
    @ApiModelProperty(value = "用户密码")
    private String userPwd;
    /*
        省略get|set
    */
}

```

- UserQuery.java

```

@ApiModel(description = "用户模块条件查询类")
public class UserQuery {
    @ApiModelProperty(value = "分页页码",example = "1")
    private Integer pageNum = 1;
    @ApiModelProperty(value = "每页大小",example = "10")
    private Integer pageSize = 10;
    @ApiModelProperty(value = "用户名")
    private String userName;
    /*
        省略get|set
    */
}

```

- ResultInfo.java

```

@ApiModel(description = "响应结果 - Model信息")
public class ResultInfo {

    @ApiModelProperty(value = "响应状态码",example = "200")

```

```
private Integer code=200;
@ApiModelProperty(value = "响应消息结果")
private String msg = "success";

@ApiModelProperty(value = "响应具体结果信息")
private Object result;
/*
  省略get|set
*/
}
```

3.4. Swagger2 接口文档访问

启动工程，浏览器访问：http://localhost:8080/springboot_mybatis/swagger-ui.html

The screenshot displays the Swagger UI for a project named 'springboot_mybatis'. The main title is '用户管理接口API文档' (User Management Interface API Documentation). The base URL is 'localhost:8080/springboot_mybatis'. The interface lists several API endpoints under the '用户模块' (User Module) section:

- POST** /user 更新用户 (Update user)
- PUT** /user 添加用户 (Add user)
- DELETE** /user/{userId} 删除用户 (Delete user)
- GET** /user/{userName} 根据用户名查询用户对象 (Query user object by username)
- GET** /user/id/{userId} 根据用户ID查询用户对象 (Query user object by user ID)
- GET** /user/list 多条件分页查询用户列表 (Query user list with multiple conditions and pagination)

The detailed view for the **GET /user/id/{userId}** endpoint is shown below:

Parameters:

Name	Description
userId * required string (path)	用户Id

Responses:

Code	Description
200	OK Example Value: { "id": 0, "userName": "string", "userPwd": "string" }
401	Unauthorized

4. SpringBoot 应用热部署

4.1. 什么是热部署?

热部署，就是在应用正在运行的时候升级软件（增加业务/修改bug），却不需要重新启动应用。

大家都知道在项目开发过程中，常常会改动页面数据或者修改数据结构，为了显示改动效果，往往需要重启应用查看改变效果，其实就是重新编译生成了新的 Class 文件，这个文件里记录着和代码等对应的各种信息，然后 Class 文件将被虚拟机的 ClassLoader 加载。

而热部署正是利用了这个特点，它监听到如果有 Class 文件改动了，就会创建一个新的 ClassLoader 进行加载该文件，经过一系列的过程，最终将结果呈现在我们眼前，Spring Boot 通过配置 DevTools 工具来达到热部署效果。

在原理上是使用了两个 ClassLoader，一个 ClassLoader 加载那些不会改变的类（第三方 Jar 包），另一个 ClassLoader 加载会更改的类，称为 restart ClassLoader，这样在有代码更改的时候，原来的 restart ClassLoader 被丢弃，重新创建一个 restart ClassLoader，由于需要加载的类比较少，所以实现了较快的重启时间。

4.2. 热部署环境配置与测试

4.2.1. 配置 DevTools 环境

- 修改 Pom 文件，添加 DevTools 依赖

```
<!-- DevTools 的坐标 -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <!--当前这个项目被继承之后，这个不向下传递-->
    <optional>true</optional>
</dependency>
```

同时在 plugin 中添加 devtools 生效标志

```
<plugin>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
    <configuration>
        <fork>true</fork><!-- 如果没有该配置，热部署的devtools不生效 -->
    </configuration>
</plugin>
```

devtools 可以实现页面热部署（即页面修改后会立即生效，这个可以直接在 application.properties 文件中配置 spring.thymeleaf.cache=false 来实现），实现类文件热部署（类文件修改后不会立即生效），实现对属性文件的热部署。即 devtools 会监听 classpath 下的文件变动，并且会立即重启应用（发生在保存时机），注意：因为其采用的虚拟机机制，该项重启是很快的。配置了后在修改 java 文件后也就支持了热启动，不过这种方式是属于项目重启（速度比较快的项目重启），会清空 session 中的值，也就是如果有用户登陆的话，项目重启后需要重新登陆。

默认情况下，/META-INF/maven, /META-INF/resources, /resources, /static, /templates, /public 这些文件夹下的文件修改不会使应用重启，但是会重新加载（devtools 内嵌了一个 LiveReload server，当资源发生改变时，浏览器刷新）

4.2.2. 全局配置文件配置

在 application.yml 中配置 spring.devtools.restart.enabled=false，此时 restart 类加载器还会初始化，但不会监视文件更新。

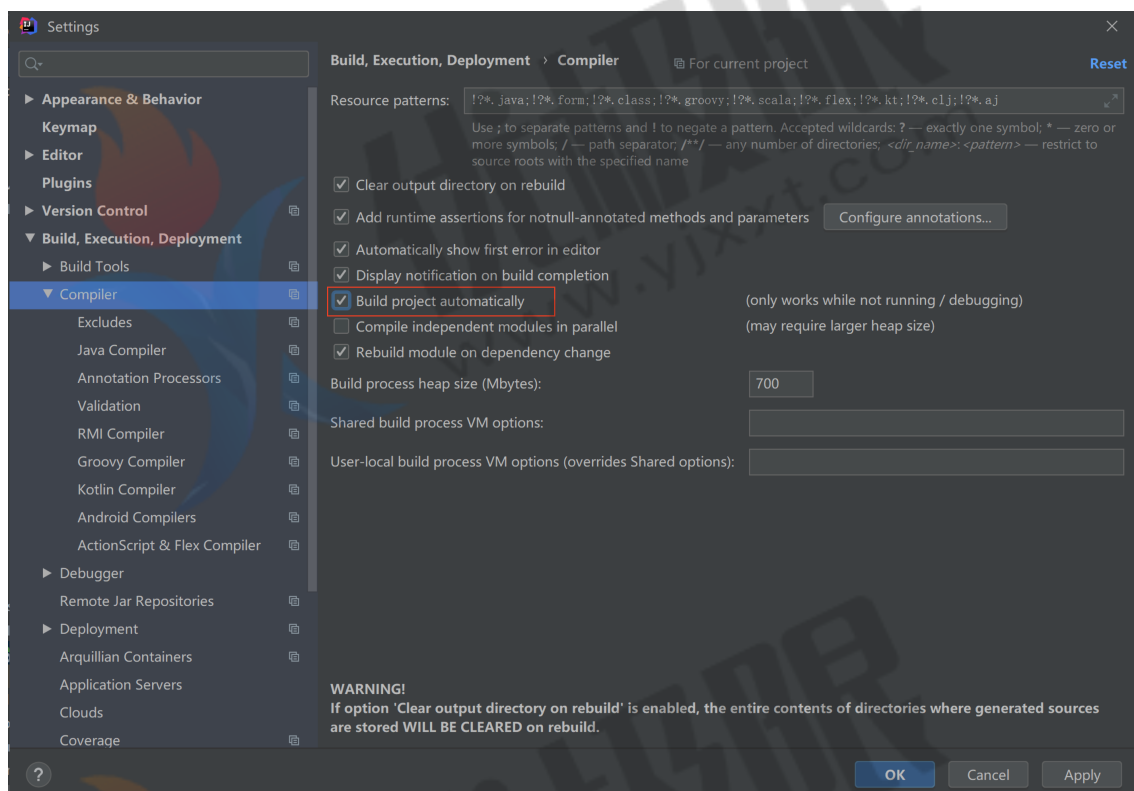
```
spring:
  ## 热部署配置
  devtools:
    restart:
      enabled: true
      # 设置重启的目录，添加目录的文件需要restart
      additional-paths: src/main/java
      # 解决项目自动重新编译后接口报404的问题
      poll-interval: 3000
      quiet-period: 1000
```

4.2.3. IDEA 配置

当我们修改了 Java 类后，IDEA 默认是不自动编译的，而 spring-boot-devtools 又是监测 classpath 下的文件发生变化才会重启应用，所以需要设置 IDEA 的自动编译。

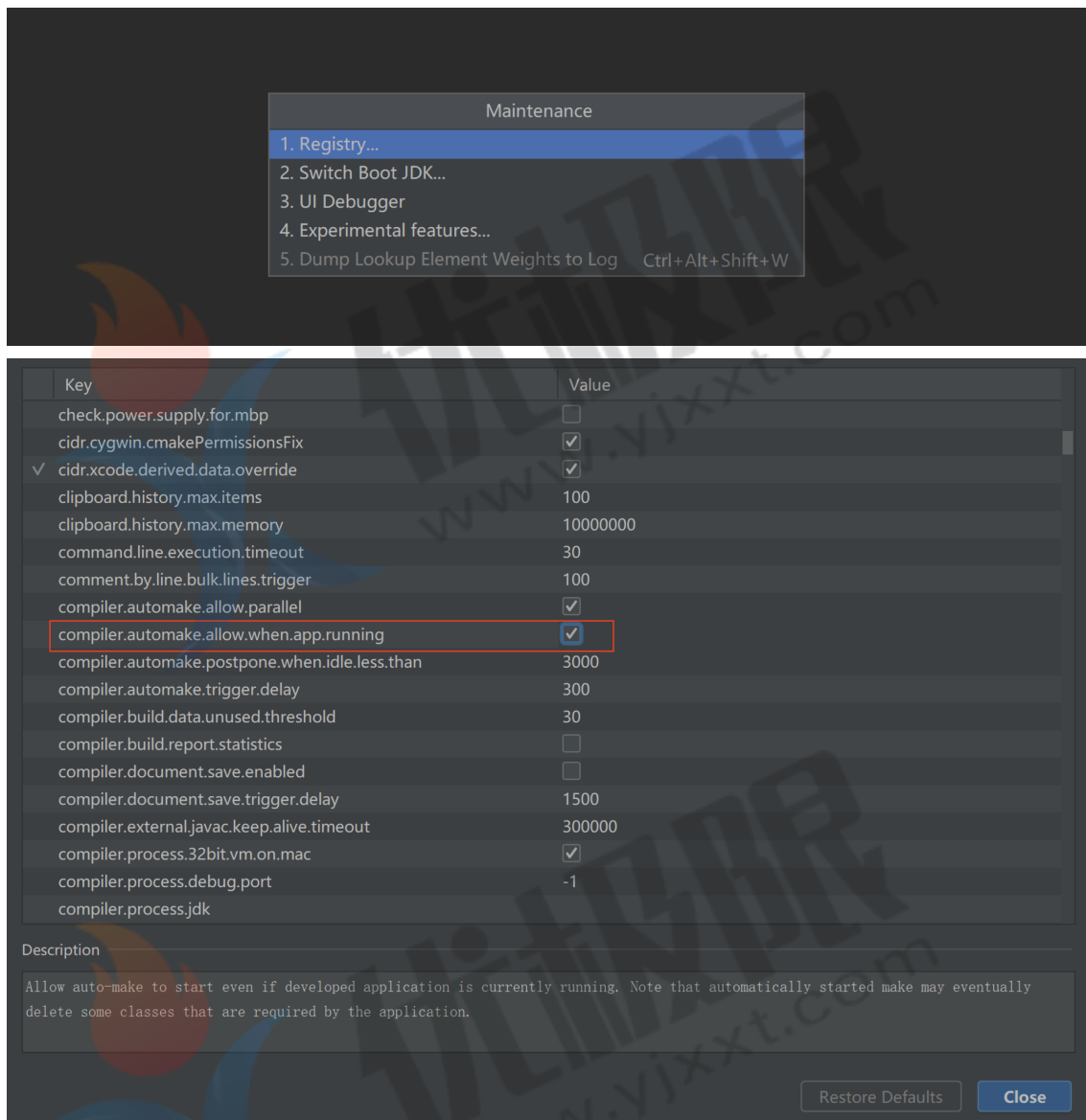
- 自动编译配置

File -> Settings -> Compiler -> Build Project automatically



- Registry 属性修改

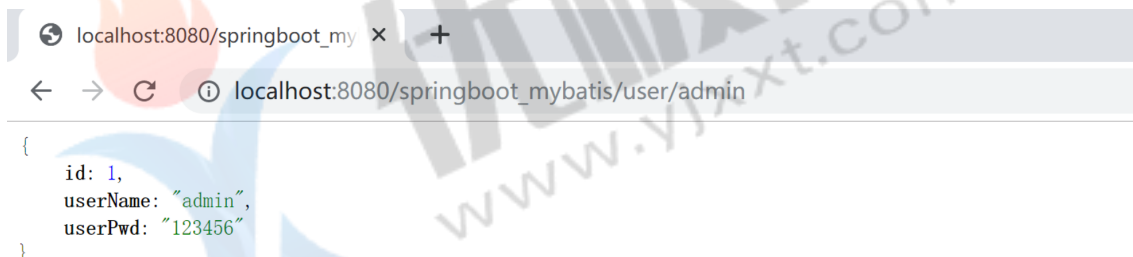
ctrl + shift + alt + /，选择 Registry，勾选 Compiler autoMake allow when app running

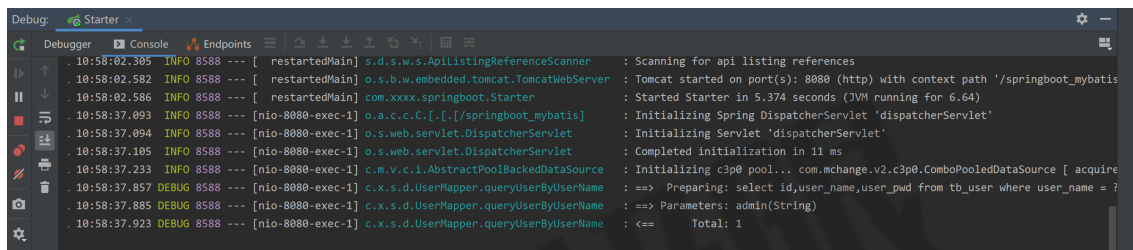


4.2.4. 热部署效果测试

- 第一次访问 `user/{userName}` 接口

```
@GetMapping("user/{userName}")
@ApiOperation(value = "根据用户名查询用户记录")
@ApiImplicitParam(name = "userName", value = "查询参数", required =
true, paramType = "path")
public User queryUserByUserName(@PathVariable String userName){
    return userService.queryUserByUserName(userName);
}
```

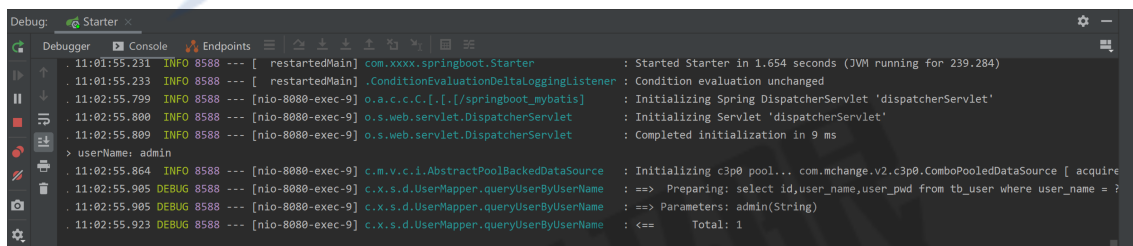
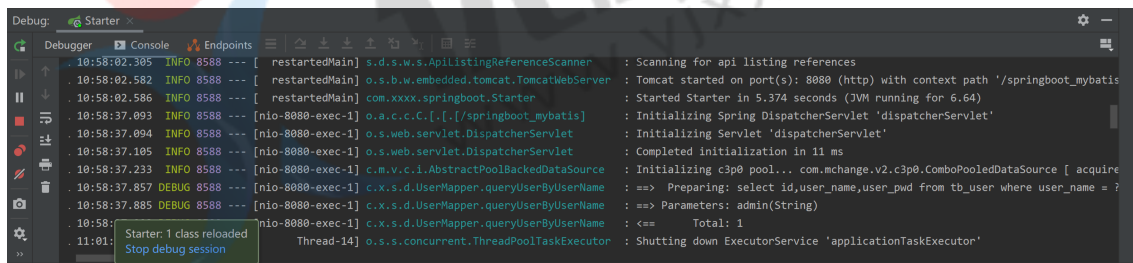
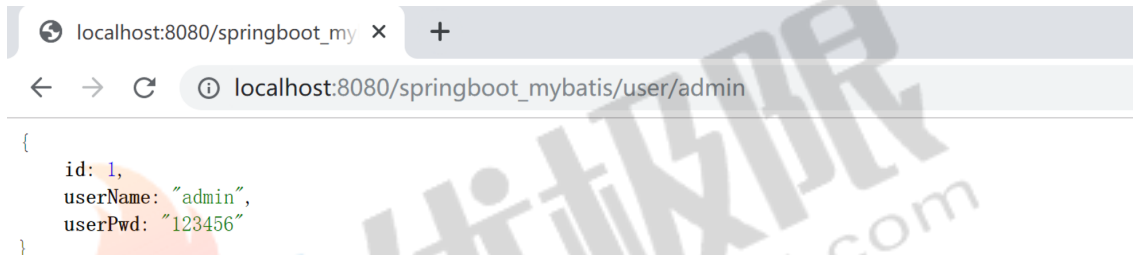




- 修改接口代码

控制台打印接收的 userName 参数, ctrl+f9 键重新编译, 浏览器访问

```
@GetMapping("user/{userName}")
@ApiOperation(value = "根据用户名查询用户记录")
@ApiImplicitParam(name = "userName", value = "查询参数", required = true, paramType = "path")
public User queryUserByUserName(@PathVariable String userName){
    System.out.println("查询参数 --> userName:"+userName);
    return userService.queryUserByUserName(userName);
}
```



5. SpringBoot 单元测试

做过 web 项目开发的对于单元测试都并不陌生了, 通过它能够快速检测业务代码功能的正确与否, SpringBoot 框架对单元测试也提供了良好的支持, 来看 SpringBoot 应用中单元测试的使用。

5.1. pom.xml 测试依赖添加

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
</dependency>
```

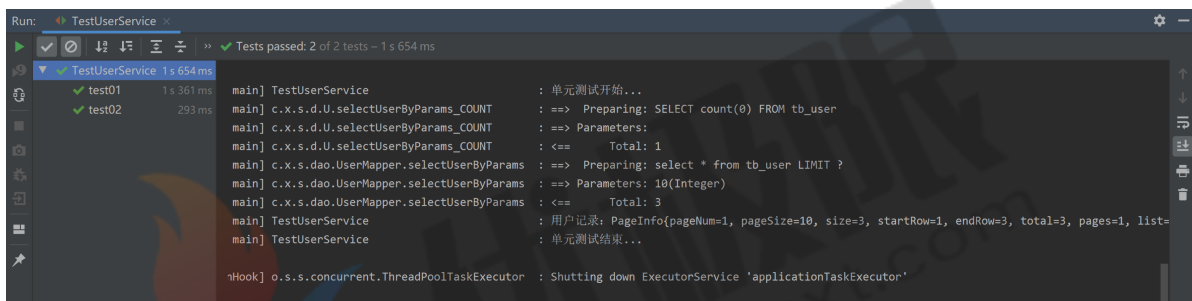
5.2. Service业务方法测试

这里以 UserService 为例，src/test/java 目录下添加测试包 com.xxx.springboot.service 定义测试类代码如下：

```
@RunWith(SpringRunner.class)
@SpringBootTest(classes = {Starter.class})
public class TestUserService {
    private Logger log = LoggerFactory.getLogger(TestUserService.class);

    @Resource
    private UserService userService;

    @Before
    public void before(){
        log.info("单元测试开始...");
    }
    @Test
    public void test01(){
        log.info("用户记录: {}", userService.queryUserById(1).toString());
    }
    @Test
    public void test02(){
        log.info("用户记录: {}", userService.queryUserByParams(new
        UserQuery()).toString());
    }
    @After
    public void after(){
        log.info("单元测试结束...");
    }
}
```



```
Run: TestUserService
Tests passed: 2 of 2 tests - 1 s 654 ms

TestUserService 1 s 654 ms
test01 1 s 361 ms
test02 293 ms

main] TestUserService : 单元测试开始...
main] c.x.s.d.U.selectUserByParams_COUNT : ==> Preparing: SELECT count(0) FROM tb_user
main] c.x.s.d.U.selectUserByParams_COUNT : ==> Parameters:
main] c.x.s.d.U.selectUserByParams_COUNT : <== Total: 1
main] c.x.s.dao.UserMapper.selectUserByParams : ==> Preparing: select * from tb_user LIMIT ?
main] c.x.s.dao.UserMapper.selectUserByParams : ==> Parameters: 10(Integer)
main] c.x.s.dao.UserMapper.selectUserByParams : <== Total: 3
main] TestUserService : 用户记录: PageInfo{pageNum=1, pageSize=10, size=3, startRow=1, endRow=3, total=3, pages=1, list=
main] TestUserService : 单元测试结束...

Hook] o.s.s.concurrent.ThreadPoolTaskExecutor : Shutting down ExecutorService 'applicationTaskExecutor'
```

5.3. 控制层接口方法测试

视图层代码使用 MockMvc 进行测试，这里以 UserController 为例，src/test/java 目录下添加测试包 com.xxxx.springboot.controller 定义测试类代码如下：

```
@RunWith(SpringRunner.class)
@SpringBootTest(classes = {Starter.class})
@AutoConfigureMockMvc
public class TestUserController {

    private Logger log = LoggerFactory.getLogger(TestUserController.class);

    @Autowired
    private MockMvc mockMvc;

    // 用户列表查询
    @Test
    public void apiTest01() throws Exception{

        // 构建请求
        MockHttpServletRequestBuilder request =
            MockMvcRequestBuilders.get("/user/list")
                .contentType("text/html") // 指定请求的contentType头信息
                .accept(MediaType.APPLICATION_JSON); // 指定请求的Accept头信息

        // 发送请求，获取请求结果
        ResultActions perform = mockMvc.perform(request);

        // 请求结果校验
        perform.andExpect(MockMvcResultMatchers.status().isOk());

        // 表示执行完成后返回相应的结果
        MvcResult mvcResult = perform.andReturn();

        // 得到执行后的响应
        MockHttpServletResponse response = mvcResult.getResponse();

        log.info("响应状态: {}", response.getStatus());
        log.info("响应内容: {}", response.getContentAsString());

    }

    // 用户名记录查询
    @Test
    public void apiTest02() throws Exception{
        MvcResult mvcResult =
            mockMvc.perform(MockMvcRequestBuilders.get("/user/admin"))
                .andExpect(MockMvcResultMatchers.status().isOk()).andReturn();
        log.info("响应状态: {}", mvcResult.getResponse().getStatus());
        log.info("响应内容: {}", mvcResult.getResponse().getContentAsString());
    }
}
```

```
Run: TestUserController x
Tests passed: 2 of 2 tests - 2 s 58 ms

main] c.m.v.c.i.AbstractPoolBackedDataSource : Initializing c3p0 pool... com.mchange.v2.c3p0.ComboPooledDataSource [ acquireIncrement -> 3, ac
main] c.x.s.d.U.selectUserByParams_COUNT : ==> Preparing: SELECT count(0) FROM tb_user
main] c.x.s.d.U.selectUserByParams_COUNT : ==> Parameters:
main] c.x.s.d.U.selectUserByParams_COUNT : <== Total: 1
main] c.x.s.dao.UserMapper.selectUserByParams : ==> Preparing: select * from tb_user LIMIT ?
main] c.x.s.dao.UserMapper.selectUserByParams : ==> Parameters: 10(Integer)
main] c.x.s.dao.UserMapper.selectUserByParams : <== Total: 3
main] TestUserController : 响应状态:200
main] TestUserController : 响应内容:{"total":3,"list":[{"id":1,"userName":"admin","userPwd":"123456"}, {"id":2,"userName":"

main] c.x.s.d.UserMapper.queryUserByUserName : ==> Preparing: select id,user_name,user_pwd from tb_user where user_name = ?
main] c.x.s.d.UserMapper.queryUserByUserName : ==> Parameters: admin(String)
main] c.x.s.d.UserMapper.queryUserByUserName : <== Total: 1
main] TestUserController : 响应状态:200
main] TestUserController : 响应内容:{"id":1,"userName":"admin","userPwd":"123456"}

mHook] o.s.s.concurrent.ThreadPoolTaskExecutor : Shutting down ExecutorService 'applicationTaskExecutor'
```

6. 分布式缓存 Ehcache 整合

EhCache 是一个比较成熟的 Java 缓存框架，最早从 hibernate 发展而来，是进程中的缓存系统，它提供了用内存，磁盘文件存储，以及分布式存储方式等多种灵活的 cache 管理方案，快速简单。

Spring Boot 对 Ehcache 的使用提供支持，所以在 Spring Boot 中只需简单配置即可使用 Ehcache 实现数据缓存处理。

6.1. Spring Cache 相关注解说明

SpringBoot 缓存实现内部使用 SpringCache 实现缓存控制，这里集成 Ehcache 实际上是对 SpringCache 抽象的其中一种实现，这里在使用 Ehcache 实现缓存控制时相关注解说明如下：

6.1.1. @CacheConfig

用于标注在类上，可以存放该类中所有缓存的公有属性，比如设置缓存的名字。

```
@CacheConfig(cacheNames = "users")
public interface UserService { ... }
```

配置了该数据访问对象中返回的内容将存储于名为users的缓存对象中，我们也可以不使用该注解，直接通过@Cacheable自己配置缓存集的名字来定义。

6.1.2. @Cacheable

应用到读取数据的方法上，即可缓存的方法，如查找方法，先从缓存中读取，如果没有再调用相应方法获取数据，然后把数据添加到缓存中。

该注解主要有下面几个参数：

- **value、cacheNames**：两个等价的参数（cacheNames 为 Spring 4 新增，作为 value 的别名），用于指定缓存存储的集合名。由于 Spring 4 中新增了 @CacheConfig，因此在 Spring 3 中原本必须有的 value 属性，也成为非必需项了。
- **key**：缓存对象存储在 Map 集合中的 key 值，非必需，缺省按照函数的所有参数组合作为 key 值，若自己配置需使用 SpEL 表达式，比如：@Cacheable(key = "#p0")：使用函数第一个参数作为缓存的 key 值，更多关于 SpEL 表达式的详细内容可参考官方文档。
- **condition**：缓存对象的条件，非必需，也需使用 SpEL 表达式，只有满足表达式条件的内容才会被缓存，比如：@Cacheable(key = "#p0", condition = "#p0.length() < 3")，表示只有当第一个参数的长度小于3的时候才会被缓存。
- **unless**：另外一个缓存条件参数，非必需，需使用 SpEL 表达式。它不同于 condition 参数的地方在于它的判断时机，该条件是在函数被调用之后才做判断的，所以它可以通过对 result 进行判断。

- **keyGenerator**: 用于指定 key 生成器，非必需。若需要指定一个自定义的 key 生成器，我们需要去实现org.springframework.cache.interceptor.KeyGenerator 接口，并使用该参数来指定。需要注意的是：该参数与 key 是互斥的。
- **cacheManager**: 用于指定使用哪个缓存管理器，非必需。只有当有多个时才需要使用
- **cacheResolver**: 用于指定使用那个缓存解析器，非必需。需通过org.springframework.cache.interceptor.CacheResolver 接口来实现自己的缓存解析器，并用该参数指定。

```
@Cacheable(value = "user", key = "#id")
User selectUserId(final Integer id);
```

6.1.3. @CachePut

应用到写数据的方法上，如新增/修改方法，调用方法时会自动把相应的数据放入缓存，@CachePut 的参数与 @Cacheable 类似，示例如下：

```
@CachePut(value = "user", key = "#user.id")
public User save(User user) {
    users.add(user);
    return user;
}
```

6.1.4. @CacheEvict

应用到移除数据的方法上，如删除方法，调用方法时会从缓存中移除相应的数据，示例如下：

```
@CacheEvict(value = "user", key = "#id")
void delete(final Integer id);
```

除了同 @Cacheable 一样的参数之外，@CacheEvict 还有下面两个参数：

- **allEntries**: 非必需，默认为 false。当为 true 时，会移除所有数据
- **beforeInvocation**: 非必需，默认为 false，会在调用方法之后移除数据。当为 true 时，会在调用方法之前移除数据。

6.1.5. @Caching

组合多个 Cache 注解使用。示例：

```
@Caching(
    put = {
        @CachePut(value = "user", key = "#user.id"),
        @CachePut(value = "user", key = "#user.username"),
        @CachePut(value = "user", key = "#user.age")
    }
)
```

将 id ---> user; username ---> user; age ---> user 进行缓存。

6.2. 环境配置

6.2.1. pom.xml 依赖添加

```
<!-- Ehcache -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-cache</artifactId>
</dependency>
<dependency>
    <groupId>net.sf.ehcache</groupId>
    <artifactId>ehcache</artifactId>
</dependency>
```

6.2.2. ehcache.xml 文件添加

src/main/resources 目录下添加 ehcache.xml 文件，内容如下：

```
<ehcache name="mycache">
    <!--
        如果不使用磁盘存储,只需要将diskStore注释掉即可;
        如果使用,需要在ehcache.xml文件中的ehcache元素下的定义一个diskStore元素并指定其
        path属性。
    -->
    <diskStore path="C:\java\cache"/>
    <!--
        name:缓存名称。
        maxElementsInMemory:缓存最大数目
        maxElementsOnDisk:硬盘最大缓存个数。
        eternal:对象是否永久有效,一但设置了,timeout将不起作用。
        overflowToDisk:是否保存到磁盘,当系统宕机时
        timeToIdleSeconds:设置对象在失效前的允许闲置时间(单位:秒)。
            仅当eternal=false对象不是永久有效时使用,可选属性,默认值是0,表示可闲置时间无穷
            大。
        timeToLiveSeconds:设置对象在失效前允许存活时间(单位:秒)。
            最大时间介于创建时间和失效时间之间。
            仅当eternal=false对象不是永久有效时使用,默认是0,也就是对象存活时间无穷大。
        diskPersistent: 是否缓存虚拟机重启期数据
            whether the disk store persists between restarts of the Virtual
            Machine.
            The default value is false.
        diskSpoolBufferSizeMB: 这个参数设置DiskStore(磁盘缓存)的缓存区大小。
            默认是30MB。每个Cache都应该有一个自己的缓冲区。
        diskExpiryThreadIntervalSeconds: 磁盘失效线程运行时间间隔,默认是120秒。
        memoryStoreEvictionPolicy: 当达到maxElementsInMemory限制时,会根据指定的策略去
            清理内存
            默认策略是LRU(最近最少使用)。你可以设置为FIFO(先进先出)或是LFU(较少使
            用)。
        clearOnFlush: 内存数量最大时是否清除。
        memoryStoreEvictionPolicy:
            可选策略有:
                LRU(最近最少使用,默认策略)
                Less Frequently Used,就是例子中使用的策略,就是一直以来最少被使用
                的。
                FIFO(先进先出)
                first in first out,这个是大家最熟的,先进先出。
                LFU(最少访问次数)
```

Least Recently Used，最近最少使用的。

缓存的元素有一个时间戳，当缓存容量满了，而又需要腾出地方来缓存新的元素的时候，

那么现有缓存元素中时间戳离当前时间最远的元素将被清出缓存。

```
-->
<defaultCache
    maxElementsInMemory="10000"
    eternal="false"
    timeToIdleSeconds="120"
    timeToLiveSeconds="120"
    maxElementsOnDisk="10000000"
    diskExpiryThreadIntervalSeconds="120"
    memoryStoreEvictionPolicy="LRU">
</defaultCache>

<cache
    name="users"
    eternal="false"
    maxElementsInMemory="100"
    overflowToDisk="false"
    diskPersistent="false"
    timeToIdleSeconds="0"
    timeToLiveSeconds="300"
    memoryStoreEvictionPolicy="LRU"/>
</ehcache>
```

6.2.3. application.yml 缓存配置

```
spring:
  datasource:
    type: com.mchange.v2.c3p0.ComboPooledDataSource
    driver-class-name: com.mysql.cj.jdbc.Driver
    url: jdbc:mysql://127.0.0.1:3306/springboot_mybatis?
    useUnicode=true&characterEncoding=utf8&serverTimezone=GMT%2B8
    username: root
    password: root
  devtools:
    restart:
      enabled: true
      # 设置重启的目录，添加目录的文件需要restart
      additional-paths: src/main/java
      # 解决项目自动重新编译后接口报404的问题
      poll-interval: 3000
      quiet-period: 1000
  ## Ehcache缓存配置
  cache:
    ehcache:
      config: classpath:ehcache.xml
```

6.2.4. 启动缓存

在 Starter 启动入口类中，添加 @EnableCaching 注解，启动缓存

```

@SpringBootApplication
@EnableCaching
@MapperScan("com.xxxx.springboot.dao")
public class Starter {

    public static void main(String[] args) {
        SpringApplication.run(Starter.class);
    }
}

```

6.2.5. JavaBean 对象实现序列化

```

@ApiModel(description = "用户实体对象")
public class User implements Serializable {
    @ApiModelProperty(value = "用户id")
    private Integer id;
    @ApiModelProperty(value = "用户名")
    private String userName;
    @ApiModelProperty(value = "用户密码")
    private String userPwd;
    /*
     省略get|set
    */
}

```

6.3. 缓存实现

这里以 UserService 的方法为例

6.3.1. 用户详情查询缓存

```

@Cacheable(value = "users",key = "#userId")
public User queryUserById(Integer userId){
    return userMapper.queryById(userId);
}

```

6.3.2. 用户列表查询缓存

```

@Cacheable(value = "users",key="#userQuery.userName+'-'+#userQuery.pageNum+'-'+#userQuery.pageSize")
public PageInfo<User> queryUserByParams(UserQuery userQuery){
    PageHelper.startPage(userQuery.getPageNum(),userQuery.getPageSize());
    return new PageInfo<User>(userMapper.selectByParams(userQuery));
}

```

6.3.3. 用户更新 & 删除缓存

```

@CacheEvict(value = "users",key="#user.id")
public void updateUser(User user) {
    AssertUtil.isTrue(StringUtils.isBlank(user.getUserName()), "用户名不能为空!");
    AssertUtil.isTrue(StringUtils.isBlank(user.getUserPwd()), "用户密码不能为空!");
    User temp = userMapper.queryUserByUsername(user.getUserName());
}

```

```

    AssertUtil.isTrue(null != temp && !(temp.getId().equals(user.getId())),"该用户已存在!");
    AssertUtil.isTrue(userMapper.update(user)<1,"用户记录添加失败!");
}

@CacheEvict(value = "users",allEntries=true)
public void deleteUser(Integer userId){
    AssertUtil.isTrue(null == userId || null == userMapper.queryById(userId),"待删除记录不存在!");
    AssertUtil.isTrue(userMapper.delete(userId)<1,"用户删除失败!");
}

```

7. 全局异常与事务控制

7.1. Spring Boot 事务支持

在使用 Jdbc 作为数据库访问技术时，Spring Boot 框架定义了基于 jdbc 的 PlatformTransactionManager 接口的实现 DataSourceTransactionManager，并在 Spring Boot 应用启动时自动进行配置。如果使用 jpa 的话 Spring Boot 同样提供了对应实现。

数据访问技术	实 现
JDBC	DataSourceTransactionManager
JPA	JpaTransactionManager
Hibernate	HibernateTransactionManager
JDO	JdoTransactionManager
分布式事务	JtaTransactionManager

这里 Spring Boot 集成了 mybatis 框架，mybatis 底层数据访问层实现基于 jdbc 来实现，所以在 Spring Boot 环境下对事务进行控制，事务实现由 Spring Boot 实现并自动配置，在使用时通过注解方式标注相关方法加入事务控制即可。

- 声明式事务配置

```

@Transactional(propagation = Propagation.REQUIRED)
public void saveUser(User user) {
    AssertUtil.isTrue(StringUtils.isBlank(user.getUserName()), "用户名不能为空!");
    AssertUtil.isTrue(StringUtils.isBlank(user.getUserPwd()), "用户密码不能为空!");
    User temp = userMapper.queryUserByUserName(user.getUserName());
    AssertUtil.isTrue(null != temp, "该用户已存在!");
    AssertUtil.isTrue(userMapper.save(user)<1,"用户记录添加失败!");
}

@Transactional(propagation = Propagation.REQUIRED)
public void updateUser(User user) {
    AssertUtil.isTrue(StringUtils.isBlank(user.getUserName()), "用户名不能为空!");
    AssertUtil.isTrue(StringUtils.isBlank(user.getUserPwd()), "用户密码不能为空!");
    User temp = userMapper.queryUserByUserName(user.getUserName());
    AssertUtil.isTrue(null != temp && !(temp.getId().equals(user.getId())),"该用户已存在!");
    AssertUtil.isTrue(userMapper.update(user)<1,"用户记录添加失败!");
}

@Transactional(propagation = Propagation.REQUIRED)

```

```
public void deleteUser(Integer id){
    AssertUtil.isTrue(null == id || null ==userMapper.queryById(id),"待删除记录不存在!");
    AssertUtil.isTrue(userMapper.delete(id)<1,"用户删除失败!");
}
```

7.2. Spring Boot 全局异常处理

SpringMvc 中对异常统一处理提供了相应处理方式, 推荐大家使用的是实现接口 `HandlerExceptionResolver` 的方式, 对代码侵入性较小。

在Spring Boot 应用中同样提供了对异常的全局性处理, 相关注解如下:

7.2.1. @ControllerAdvice

该注解组合了 `@Component` 注解功能, 最常用的就是作为全局异常处理的切面类, 同时通过该注解可以指定包扫描的范围。`@ControllerAdvice` 约定了几种可行的返回值, 如果是直接返回 `model` 类的话, 需要使用 `@ResponseBody` 进行 json 转换

7.2.2. @ExceptionHandler

该注解在 Spring 3.X 版本引入, 在处理异常时标注在方法级别, 代表当前方法处理的异常类型有哪些具体应用以 Restful 接口为例, 测试保存用户接口。

7.2.3. 全局异常应用

7.2.3.1. 异常抛出与全局异常捕获

- UserController 查询接口

```
@ApiOperation(value = "根据用户id查询用户记录")
@ApiImplicitParam(name = "userId",value = "用户ID",required = true, paramType = "path")
@GetMapping("user/id/{userId}")
public User queryUserById(@PathVariable Integer userId){
    return userService.queryUserById(userId);
}
```

- UserService 查询业务方法, 抛出 `ParamExceptions` 异常

```
public User queryUserById(Integer userId){
    // 抛出异常
    AssertUtil.isTrue(true,"异常测试...");
    return userMapper.queryById(userId);
}
```

- 全局异常处理类 `GlobalExceptionHandler` 定义

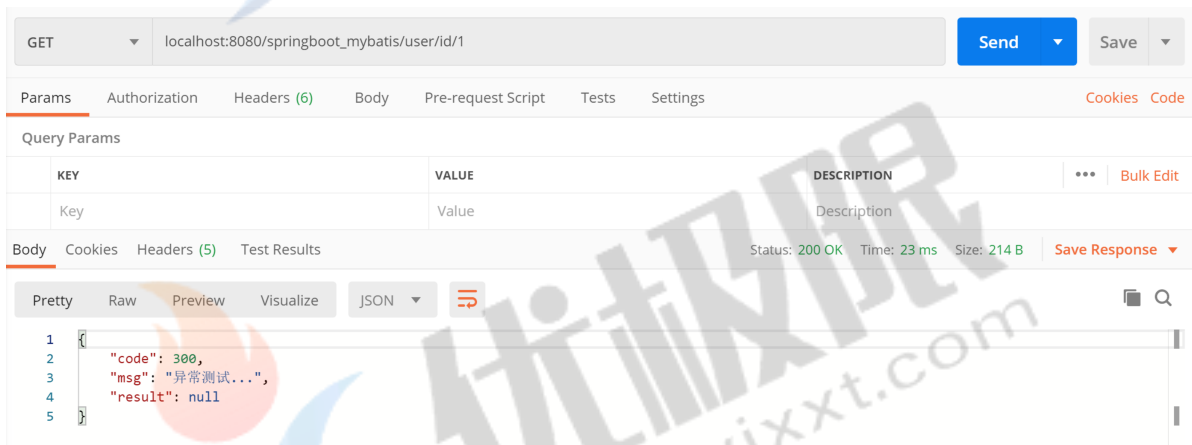
```
@ControllerAdvice
public class GlobalExceptionHandler{
    /**
     * 全局异常处理 返回json
     * @param e
```

```

    * @return
    */
    @ExceptionHandler(value = Exception.class)
    @ResponseBody
    public ResultInfo exceptionHandler(Exception e){
        ResultInfo resultInfo = new ResultInfo();
        resultInfo.setCode(300);
        resultInfo.setMsg("操作失败!");
        if(e instanceof ParamsException){
            ParamsException pe = (ParamsException) e;
            resultInfo.setMsg(pe.getMsg());
            resultInfo.setCode(pe.getCode());
        }
        return resultInfo;
    }
}

```

- Postman 执行测试效果



7.2.3.2. 特定异常处理

- 通过 `@ExceptionHandler` 标注方法处理特定异常，这里以用户未登录异常为例，通过全局异常进行处理

```

/**
 * 用户未登录异常特殊处理 返回json
 * @param authExceptions
 * @return
 */
@ExceptionHandler(value = NoLoginException.class)
@ResponseBody
public ResultInfo userNotLoginHandler(NoLoginException authExceptions){
    System.out.println("用户未登录异常处理。。。");
    ResultInfo resultInfo = new ResultInfo();
    resultInfo.setCode(authExceptions.getCode());
    resultInfo.setMsg(authExceptions.getMsg());
    return resultInfo;
}

```

- 在用户修改接口中抛出未登录异常为例进行测试

```

/**
 * 修改用户
 * @param user

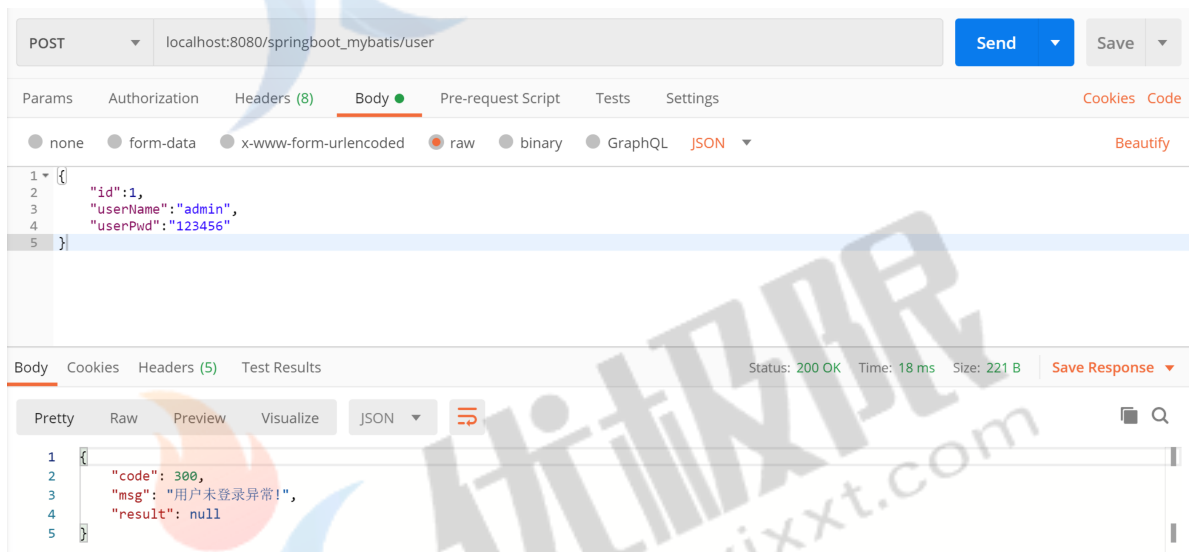
```

```

    * @return
    */
    @ApiOperation(value = "更新用户")
    @ApiImplicitParam(name = "user", value = "用户对象")
    @PostMapping("/user")
    public ResultInfo updateUser(@RequestBody User user) {
        if(1 == 1){
            throw new NoLoginException();
        }
        ResultInfo resultInfo = new ResultInfo();
        userService.updateUser(user);
        return resultInfo;
    }

```

- Postman 执行测试效果



8. SpringBoot 数据校验 - Validation

日常项目开发中，对于前端提交的表单，后台接口接收到表单数据后，为了程序的严谨性，通常后端会加入业务参数的合法校验操作来避免程序的非技术性 bug，这里对于客户端提交的数据校验，SpringBoot 通过 `spring-boot-starter-validation` 模块包含了数据校验的工作。

这里主要介绍 Spring Boot 中对请求数据进行校验，相关概念如下

- JSR303：JSR303 是一项标准，只提供规范不提供实现，规定一些校验规范即校验注解，如 `@Null`，`@NotNull`，`@Pattern`，位于 `javax.validation.constraints` 包下。JSR-349 是其升级版，添加了一些新特性。
- Hibernate Validation：Hibernate Validation 是对这个规范的实现，并增加了一些其他校验注解，如 `@Email`，`@Length`，`@Range` 等等。
- Spring Validation：Spring Validation 对 Hibernate Validation 进行了二次封装，在 Spring MVC 模块中添加了自动校验，并将校验信息封装进了特定的类中。

8.1. 环境配置

实现参数校验，程序必须引入 `spring-boot-starter-validation` 依赖，只是在引入 `spring-boot-starter-web` 依赖时，该模块会自动依赖 `spring-boot-starter-validation`，所以程序中引入 `spring-boot-starter-web` 会一并依赖 `spring-boot-starter-validation` 到项目中。

```

▶ Maven: org.springframework.boot:spring-boot-starter-json:2.2.2.RELEASE
▶ Maven: org.springframework.boot:spring-boot-starter-logging:2.2.2.RELEASE
▶ Maven: org.springframework.boot:spring-boot-starter-quartz:2.2.2.RELEASE
▶ Maven: org.springframework.boot:spring-boot-starter-test:2.2.2.RELEASE
▶ Maven: org.springframework.boot:spring-boot-starter-tomcat:2.2.2.RELEASE
▶ Maven: org.springframework.boot:spring-boot-starter-validation:2.2.2.RELEASE
▶ Maven: org.springframework.boot:spring-boot-starter-web:2.2.2.RELEASE
▶ Maven: org.springframework.boot:spring-boot-test:2.2.2.RELEASE

```

8.2. 校验相关注解

注解	功能
@AssertFalse	可以为null,如果不为null的话必须为false
@AssertTrue	可以为null,如果不为null的话必须为true
@DecimalMax	设置不能超过最大值
@DecimalMin	设置不能超过最小值
@Digits	设置必须是数字且数字整数的位数和小数的位数必须在指定范围内
@Future	日期必须在当前日期的未来
@Past	日期必须在当前日期的过去
@Max	最大不得超过此最大值
@Min	最大不得小于此最小值
@NotNull	不能为null, 可以是空
@Min	最大不得小于此最小值
@Pattern	必须满足指定的正则表达式
@Size	集合、数组、map等的size()值必须在指定范围内
@Email	必须是email格式
@Length	长度必须在指定范围内
@NotBlank	字符串不能为null,字符串trim()后也不能等于""
@NotEmpty	不能为null, 集合、数组、map等size()不能为0; 字符串trim()后可以等于""
@Range	值必须在指定范围内
@URL	必须是一个URL

8.3. 校验注解使用

- User 实体类参数校验注解

```

public class User implements Serializable {

    private Integer id;

```

```

@NotBlank(message = "用户名不能为空!")
private String userName;

@NotBlank(message = "用户密码不能为空!")
@Length(min = 6, max = 10, message = "密码长度至少6位但不超过10位!")
private String userPwd;

>Email
private String email;

/*
    省略get set 方法
*/
}

```

- 接口方法形参 @Valid 注解添加

```

@PostMapping("user")
@ApiOperation(value = "用户添加")
@ApiImplicitParam(name = "user", value = "用户实体类", dataType = "User")
public ResultInfo saveUser02(@Valid User user){
    ResultInfo resultInfo = new ResultInfo();
    // userService.saveUser(user);
    return resultInfo;
}

```

- 全局异常错误信息捕捉

```

/**
 * 全局异常处理 返回json
 * @param e
 * @return
 */
@ExceptionHandler(value = Exception.class)
@ResponseBody
public ResultInfo exceptionHandler(Exception e){
    ResultInfo resultInfo = new ResultInfo();
    resultInfo.setCode(300);
    resultInfo.setMsg("操作失败!");
    if(e instanceof ParamsException){
        ParamsException pe = (ParamsException) e;
        resultInfo.setMsg(pe.getMsg());
        resultInfo.setCode(pe.getCode());
    } else if(e instanceof BindException){
        BindException be = (BindException) e;
        resultInfo.setResult(be.getBindingResult().getFieldError().getDefaultMessage());
    }
    return resultInfo;
}

```

- PostMan 接口测试

PUT localhost:8080/springboot_mybatis/user02?userName=test&userPwd=1234 Send Save

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies Code

Query Params

	KEY	VALUE	DESCRIPTION	...	Bulk Edit
☒	userName	test			
☒	userPwd	1234			
	Key	Value	Description		

Body Cookies Headers (5) Test Results Status: 200 OK Time: 13 ms Size: 239 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "code": 500,
3   "msg": "密码长度至少6位且不超过10位!",
4   "result": null
5 }
```

PUT localhost:8080/springboot_mybatis/user02?userName=test&userPwd=123456&email=123 Send Save

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies Code

Query Params

	KEY	VALUE	DESCRIPTION	...	Bulk Edit
☒	userName	test			
☒	userPwd	123456			X
☒	email	123			
	Key	Value	Description		

Body Cookies Headers (5) Test Results Status: 200 OK Time: 328 ms Size: 238 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "code": 500,
3   "msg": "不是一个合法的电子邮件地址",
4   "result": null
5 }
```